Projet de Moteur de Recherche
M1 Programmation Spécialisée

Laura Moret

8 novembre 2026

https://github.com/jortanix/projet_prg_spe

Table des matières

1 Spécifications **3**

2 Analyse **3**

2.1 Environnement de travail 3

2.2 Données identifiées 4

2.3 Diagramme de classes 4

3 Conception **4**

3.1 Répartition des tâches 4

3.2 Algorithmes spécifiques 5

3.2.1 Analyse textuelle 5

3.2.2 Score TF-IDF 5

3.2.3 Recherche et similarité 5

3.2.4 Clustering 5

3.3 Problèmes rencontrés et solutions 5

3.4 Exemple d'utilisation 6

4 Validation **6**

4.1 Tests unitaires 6

4.2 Tests globaux 6

5 Maintenance et perspectives **7**

5.1 Évolutions en cours 7

5.2 Perspectives d'amélioration 7

6 Contraintes techniques **7**

7 Conclusion **7**

Résumé

Ce projet porte sur la conception et l'implémentation d'un moteur de recherche capable d'analyser, de structurer et d'interroger un corpus de documents textuels issus de deux sources distinctes : Reddit et arXiv. Il s'appuie sur un modèle orienté objet, un système de scoring TF-IDF calculé manuellement, ainsi qu'une série de fonctionnalités avancées (recherche multi-critères, clustering, recommandations, analyse statistique et export des résultats).

Le travail est organisé en trois parties. La première établit la structure de données (documents, auteurs, corpus avec patron singleton) et l'acquisition des données. La deuxième ajoute l'analyse textuelle et le moteur de recherche basé sur la matrice Documents × Termes. La troisième, en cours de finalisation, apporte les fonctionnalités avancées et une future interface interactive.

1 Spécifications

L'objectif est de construire un moteur de recherche complet permettant de :

- Collecter et structurer des documents textuels depuis des sources externes (Reddit et arXiv) ;
- Analyser le texte : tokenization, suppression des mots vides, construction d'un vocabulaire et calcul de fréquences ;
- Construire une matrice Documents × Termes à partir de scores TF-IDF calculés manuellement ;
- Offrir un moteur de recherche basé sur ces scores, avec filtres et tris ;
- Proposer des fonctionnalités avancées : clustering, recommandations, analyse de fréquence, tendances temporelles, statistiques et export CSV/JSON ;
- Respecter les contraintes techniques imposées (interdiction de scikit-learn et nltk, calculs effectués manuellement).

Le périmètre de la partie 3 prévoit également une interface graphique sous forme de notebook Jupyter utilisant `ipywidgets`. Cette interface n'étant pas encore intégrée au moment de la rédaction, elle est présentée dans les perspectives.

2 Analyse

2.1 Environnement de travail

Le projet est développé en Python 3.x. Les bibliothèques utilisées sont limitées aux besoins strictement nécessaires :

- NumPy : calculs numériques et manipulation de vecteurs ;
- SciPy : matrices creuses (`csr_matrix`) et algorithme de clustering K-means ;
- Pandas : organisation des données et export en CSV ;
- Matplotlib et Seaborn : visualisation des résultats (facultatif) ;
- Bibliothèque standard : `datetime`, `collections`, `re`, `json`.

2.2 Données identifiées

Deux types de documents sont traités, modélisés sous forme de classes dérivées :

- **Documents Reddit** : titre, auteur, date, URL, texte et nombre de commentaires ;
- **Documents arXiv** : titre, auteur, date, URL, texte et liste de co-auteurs.

Chaque document appartient à un **auteur**, et l'ensemble des documents et des auteurs constitue un **corpus**. Le corpus conserve un vocabulaire global ainsi que la matrice Documents × Termes.

2.3 Diagramme de classes

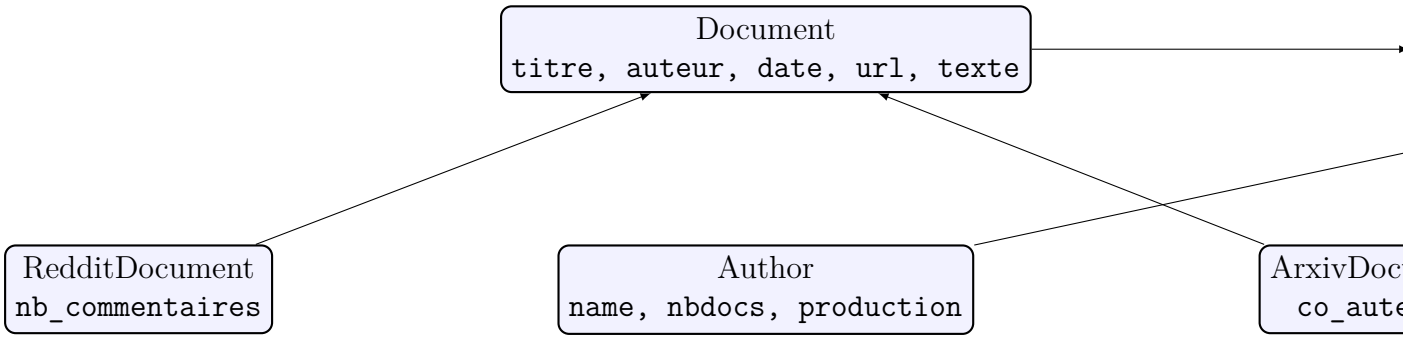


FIGURE 1 – Diagramme de classes simplifié du projet

La classe `Corpus` centralise la recherche, la construction de la matrice et la persistance, tandis que `SearchEngine` encapsule les fonctionnalités avancées (recherche multi-critères, clustering, recommandations, statistiques et export).

3 Conception

3.1 Répartition des tâches

Le projet a été réalisé de manière individuelle. Le travail a été décomposé en trois étapes :

- **Partie 1 – Structure** : création des classes `Document`, `RedditDocument`, `ArxivDocument`, `Author` et `Corpus`, acquisition des données et persistance (sauvegarde en TSV) ;
- **Partie 2 – Recherche** : analyse textuelle (tokenization, suppression des mots vides), construction du vocabulaire et de la matrice Documents × Termes, calcul des scores TF-IDF, recherche et similarité entre documents ;
- **Partie 3 – Avancé** : moteur de recherche à filtres et tris, clustering, recommandations, analyse de fréquence, tendances temporelles, statistiques, export CSV/JSON, et (en perspective) interface `ipywidgets`.

3.2 Algorithmes spécifiques

3.2.1 Analyse textuelle

L'analyse consiste en :

- la tokenization du texte (découpage en termes, gestion de la ponctuation) ;
- la suppression des mots vides et la normalisation en minuscules ;
- la construction d'un vocabulaire global avec les fréquences des termes.

3.2.2 Score TF-IDF

Le poids d'un terme t dans un document d est :

$$TF\text{-}IDF(t, d) = TF(t, d) \times \log\left(\frac{N}{DF(t)}\right)$$

où N est le nombre total de documents et $DF(t)$ le nombre de documents contenant le terme. Les scores sont stockés dans une matrice creuse Documents × Termes pour économiser la mémoire.

3.2.3 Recherche et similarité

La pertinence d'un document pour une requête est évaluée en agrégeant les scores TF-IDF des termes de la requête. La similarité entre deux documents repose sur le cosinus de leurs vecteurs.

3.2.4 Clustering

Le clustering repose sur l'algorithme **K-means** appliqué aux vecteurs de documents issus de la matrice TF-IDF (via SciPy), puis à l'attribution de chaque document au centroïde le plus proche.

3.3 Problèmes rencontrés et solutions

- **Gestion des dates** : comparaison et tri par date nécessitaient un format cohérent ; `datetime` a été utilisé et des filtres de période (`date_start`, `date_end`) ont été ajoutés à la recherche avancée.
- **Efficacité de la matrice** : la matrice Documents × Termes pouvant être volumineuse et creuse, elle est représentée en *matrice creuse* (SciPy) afin de limiter l'empreinte mémoire.
- **Héritage et polymorphisme** : les deux types de documents partagent un comportement commun ; l'héritage de `Document` et la redéfinition de `getType()` évitent la duplication de code.
- **Unicité du corpus** : le patron *singleton* garantit qu'une seule instance de corpus existe à travers l'application.

— **Caractères accentués** : la gestion de l'encodage UTF-8 (accents français) a nécessité une configuration adaptée de l'environnement Python et des fichiers de sauvegarde.

3.4 Exemple d'utilisation

Le fonctionnement du moteur s'illustre par l'enchaînement suivant :

1. Création du corpus puis initialisation automatique de son moteur de recherche ;
2. Recherche avancée : la requête est passée avec éventuellement des filtres (auteur, type, période) et un critère de tri (pertinence, date ou auteur) ;
3. Regroupement des documents similaires en un nombre donné de clusters ;
4. Obtention de recommandations basées sur la similarité avec un document de référence ;
5. Export des résultats de recherche aux formats CSV ou JSON, et consultation de statistiques globales (nombre de documents, termes les plus fréquents, types de documents).

4 Validation

4.1 Tests unitaires

Des tests unitaires couvrent chacune des fonctionnalités :

— **Partie 1** : initialisation du corpus et de ses attributs, patron singleton, création des documents Reddit et arXiv, gestion des auteurs, ajout de documents, affichage (`__repr__`) et sauvegarde ;
— **Partie 2** : construction du vocabulaire, dimensions et type de la matrice Documents × Termes, résultats de recherche, calcul de similarité et extraction des termes importants ;
— **Partie 3** : initialisation du moteur de recherche, recherche avancée (filtres, tri), clustering, recommandations, analyses (fréquence, tendances, statistiques) et export en CSV et JSON.

Chaque test renvoie un résultat explicite, et un compteur agrège le nombre de tests réussis et échoués. Des cas d'exception sont également couverts afin de vérifier le comportement en présence de données invalides.

4.2 Tests globaux

Les tests globaux vérifient l'intégration complète du système, c'est-à-dire :

— l'acquisition et la persistance des données (sauvegarde et rechargement du corpus) ;
— la chaîne complète analyse → vocabulaire → matrice TF-IDF ;
— l'enchaînement recherche → filtres → tri → export ;
— la cohérence entre le clustering, les recommandations et les statistiques produites à partir du même corpus.

5 Maintenance et perspectives

5.1 Évolutions en cours

Le développement se poursuit dans deux directions principales :

- l'**interface graphique** de la partie 3, sous forme de notebook Jupyter avec `ipywidgets` et barres de progression, permettant d'interroger et de visualiser le corpus de manière interactive ;
- l'enrichissement du **moteur de recherche** avec de nouvelles fonctionnalités bonus déjà implémentées (filtres, tris, clustering, recommandations, statistiques, export).

5.2 Perspectives d'amélioration

Les axes d'amélioration envisagés sont :

- le traitement avancé du langage (lemmatisation, stemming) pour améliorer la qualité du vocabulaire ;
- l'optimisation des performances sur de grands volumes de données (parallélisation, index inversé) ;
- le raffinement du clustering (choix automatique du nombre de clusters, autres métriques de distance) ;
- la prise en charge de formats et de sources de documents supplémentaires ;
- des visualisations avancées et interactives intégrées à l'interface.

6 Contraintes techniques

Le projet respecte les contraintes imposées :

- **Interdits** : scikit-learn, nltk et autres bibliothèques d'apprentissage ou de traitement avancé ;
- **Autorisés** : NumPy, SciPy (matrices creuses et K-means), Pandas ;
- **Calcul manuel** : l'analyse textuelle, le vocabulaire, les fréquences et les scores TF-IDF sont implémentés manuellement ;
- **Persistance** : sauvegarde du corpus au format TSV et du vocabulaire dans un fichier séparé.

7 Conclusion

Ce projet a permis de concevoir un moteur de recherche complet, structuré autour de principes orientés objet et reposant sur un calcul manuel du TF-IDF. Il intègre un moteur de recherche à filtres et tris ainsi que des fonctionnalités avancées (clustering, recommandations, statistiques et export), tout en respectant les contraintes techniques fixées. La poursuite du travail porte principalement sur l'interface graphique interactive de la partie 3.

Lien vers le dépôt

Le code source complet est disponible à l'adresse :

https://github.com/jortanix/projet_prg_spe

Références

[1] arXiv API Documentation. https://arxiv.org/help/api/index
[2] Reddit API Documentation. https://www.reddit.com/dev/api/
[3] Scipy Documentation. https://docs.scipy.org/
[4] Pandas Documentation. https://pandas.pydata.org/
[5] TF-IDF Wikipedia. https://fr.wikipedia.org/wiki/TF-IDF